Final Project: Tracing and Verifying AI Agent Tool Calls

Team project: two students per team

1 Project Overview

Build a system that captures, reconstructs, visualizes, and verifies the tool-calling behavior of an AI agent. Use **Hermes Agent** or **OpenClaw** as the agent platform. Your system must show which tool calls actually occurred when an agent answered a question, their order or causal relationships, and their outcomes. The project is an implementation and evaluation task, not merely a diagram of an intended workflow.

Work in a team of exactly two students. Both students are responsible for understanding the entire system, testing it, writing the report, and presenting it. Teams receive the same project score when their documented contributions are substantially equal. The instructor may assign different individual scores when evidence shows unequal contributions.

2 Required Outputs at a Glance

1. Run five required cases: M0–M2 again, plus F1 and F2. Add a run if none shows at least two actual tool calls.
2. Use one documented command to turn each saved run and reference record into a machine-readable graph, a readable visualization, and a validation report.
3. Check every graph against independent execution evidence; show that the validator detects a deliberately altered graph.
4. Submit the technical report, editable report-source link, slides, and reproducible GitHub repository as described below.

Items 2 and 3 describe the full-credit automated pathway. Teams that make a substantial, documented automation attempt but cannot complete it may use the reduced-credit manual fallback in Section 7. The automated pathway is the only path to an A.

3 Learning Objectives

By the end of the project, you should be able to:

1. Explain where tool calls occur in an agent run and what evidence is available to observe them.
2. Design and implement a reproducible trace capture and graph construction method.
3. Distinguish observed call order from inferred causal or data dependencies.
4. Evaluate graph completeness and correctness against an independent source of evidence.
5. Communicate the design, results, limitations, and team contributions in a professional technical report and presentation.

4 Platform and Project Scope

Choose **one** platform: Hermes Agent (Nous Research) or OpenClaw. Record the installed version or commit, model/provider, operating environment, tool configuration, and tracing interface used. Use real agent executions. A hand-written list of expected calls or a simulated graph alone does not satisfy the full-credit implementation requirement. The manual fallback permits hand-constructed digital graphs only under the requirements and grade ceiling in Section 7. A deterministic mock tool is allowed for a controlled test, but the selected agent must actually invoke it.

Begin with official platform documentation and identify a supported observation point. For example, Hermes documents `pre_tool_call` and `post_tool_call` hooks; OpenClaw documents trajectory bundles, audit records, and OpenTelemetry signals. These are examples, not required implementation choices. Explain the coverage and limitations of the interface you choose. If it omits a type of tool action, supplement it with another observable source and document the reconciliation rule. Official starting points appear in Section 12.

5 Shared Starter Dataset and Questions

Use the synthetic files and questions in `../tooltracking_starter/README.md`. Copy its `data/` folder into your controlled workspace and include the same files in your GitHub repository. **Rerun M0–M2 as new agent runs** using the same question wording as the midterm's first attempt, with `DATA_DIR` replaced by the current path. Also run **F1 and F2** at least once. F0 and F3 are optional examples. These five required cases cover a no-tool control, simple and cross-file lookups, a three-source investigation, and a missing-file case. Add further runs or your own questions when the required cases do not elicit two actual tool calls or another required behavior. Document all prompt revisions. Compare midterm and final observations where useful, but do not assume the same prompt yields the same calls twice. Validate each final graph against its own independent execution record, not the answer key, the agent's narrative, or a saved midterm trace.

Illustrative tool-call graphs

Figure 1 shows possible graph shapes for the required tests and a retry. Each box represents a separate *observed* tool invocation, even when the same tool is called again. Arrows in this example mean *next observed call in this run*; they do not claim that one call caused another. The question and final answer can appear in the graph caption or accompanying run record. These are examples only: a real agent may combine file reads into one call, repeat a call, or choose a different order. Your graph must reflect the actual saved run.

Fictional branching investigation

Consider the question: *Which device owner, if any, meets alert rule R-1?* Imagine an agent with five example tools: `read_auth_events`, `read_alert_rule`, `read_devices`, `evaluate_rule`, and `match_owner`. These tool names and the run in Figure 2 are fictional teaching examples; they are not built-in Hermes or OpenClaw tools and are not an additional required test.

In this hypothetical run, C1 reads the events, C2 reads the rule, and C3 reads the device records. C4 uses the outputs of C1 and C2 to evaluate the rule. C5 uses the output of C4 and the device records from C3 to identify the owner. Each arrow in Figure 2 means *output used as input*, not next observed call. The three reads could occur in any order; the arrows do not establish parallel execution.

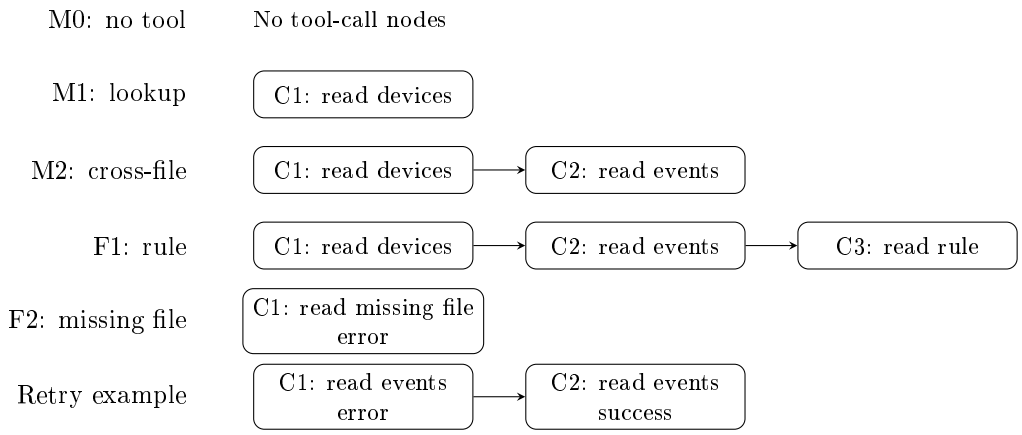


Figure 1: Illustrative call graphs. The retry uses two distinct nodes for the same tool; it is not a required outcome for F1 or F2. A refused F2 request could have no tool-call node.

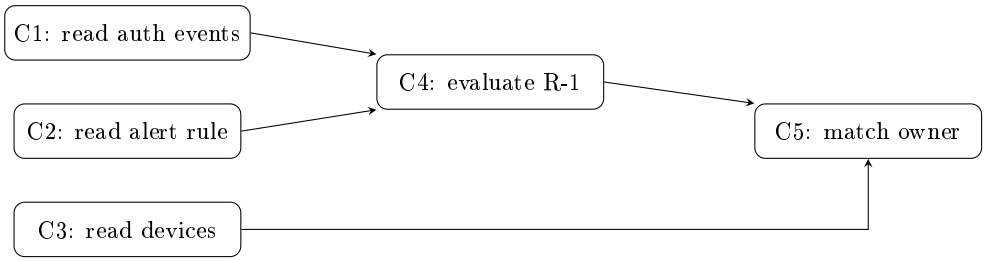


Figure 2: Hypothetical data-dependency graph. Its branching shape is not required of student runs. Show a dependency edge only when evidence establishes that one call's output was used by another.

This example shows why a complex tool-call graph can have branches and joins. A linear graph is also correct when that is what the observed calls and supported relationships show. Do not add dependency edges merely because a workflow diagram suggests them.

Fictional retry loop and write step

Suppose the first event query returns too few records to decide whether R-1 applies. In an example workflow, an `evaluate_rule` tool reports missing evidence, the agent calls `read_auth_events` again with a wider time window, evaluates the rule again, and then uses a fictional `write_report` tool to save the conclusion in a disposable workspace. These are illustrative tool choices, not required tools or an additional graded case.

Figure 3 distinguishes a *workflow loop* from the graph of actual call instances. The dashed arrow in the top panel means the agent may return to the read step when evidence is incomplete. The lower panel unrolls one possible run: C1 and C3 call the same read tool, but they are different invocations. Its solid arrows mean *next observed call*, as in Figure 1. A real graph must use the observed calls and supported edge relationships; it must not draw a self-loop on one invocation merely because the tool was used again.

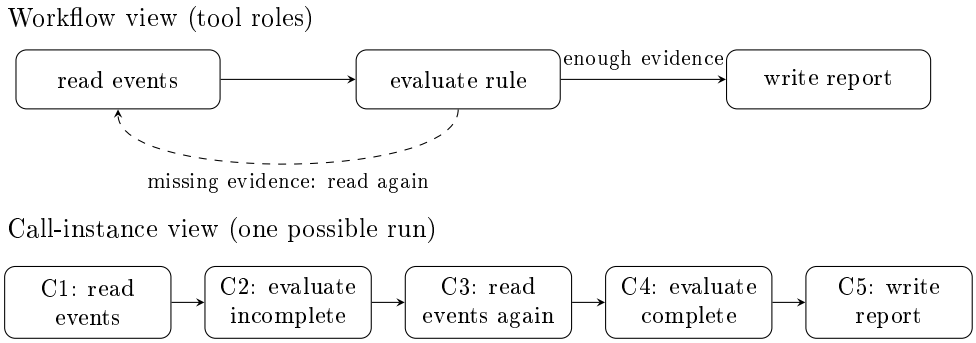


Figure 3: A conceptual workflow loop (top) and one possible sequence of distinct tool-call instances (bottom). The write tool is fictional and is not required for student implementations.

6 Required Technical Work

The full-credit pathway has three core tasks: **capture the calls, build the graph, and verify the graph**. First make the three tasks work for one run; then apply the same automated process to all required runs. Section 7 provides a reduced-credit fallback when a substantial automation attempt remains unsuccessful.

6.1 Task 1: Capture the tool calls

Run the agent and save evidence showing which tools it actually called. Implement a runnable collector or parser. For every call, preserve at least:

- run ID and unique call ID (native or generated),
- tool name,
- event order or timestamp,
- outcome, and
- a pointer to the supporting raw evidence.

Capture request/result or start/end events when the platform provides them. Save a second, independent record for Task 3. Record an unavailable field as *unknown*; do not invent timestamps, call IDs, outcomes, or dependencies. Redact secrets and personal information from shared artifacts while preserving the evidence needed to identify and order calls.

Brief design note. Describe the tracing problem, your proposed architecture, the platform observation point you selected, and why you selected it. Define the boundary of one run and what counts as a tool call. State how your method treats failed, blocked, repeated, nested, and parallel calls. Include a data-flow diagram from agent event to saved trace, graph, and validation result. Describe at least one alternative capture method you considered.

Hint. Start with one simple tool-using question. Look for an official platform trace, hook, trajectory, audit record, or instrumentation point around the tool boundary. Automating agent launch and prompt execution is optional; reliably processing saved run evidence is required.

6.2 Task 2: Build the tool-call graph

For the full-credit pathway, create one documented command that reads a saved trace and produces:

- a machine-readable graph, such as JSON, GraphML, or DOT, and
- a readable graph image or interactive view.

The command must also accept the independent record used in Task 3. It must work on a new saved run without editing the code or drawing the graph manually.

Every observed invocation must be a separate node. Show its tool name, call ID or order, and success, error, or blocked status. Repeated uses of the same tool must have different nodes and call IDs; do not merge them or replace them with a self-loop. Label a call as a retry only when the evidence supports that relationship. Define what every edge means. If you use more than one edge meaning, give them different labels or visual styles. Include the user's question and the agent's final response in the graph or its accompanying run record.

Hint. The simplest useful edge meaning is often *next observed call in this run.* Such an edge shows order, not causation. Add parent/child, retry, or data-dependency edges only when the saved evidence supports them, and do not force an order on calls that were truly concurrent.

6.3 Task 3: Verify the graph

For the full-credit pathway, extend the same command so that it compares the generated graph with the independent record and produces a validation report. The independent record may come from a separate platform audit or trajectory stream, instrumentation around the tool boundary, or a manually annotated raw trace reviewed by both team members. Explain why it is independent of the graph generator and how it can reveal missing or extra calls.

For each run, check that:

- every actual call appears exactly once and no call is fabricated,
- tool name, call ID, run association, and outcome match,
- every edge follows its declared meaning and agrees with the evidence, and
- no-tool, failed or blocked, repeated, and parallel behavior is represented correctly when it occurs.

Report counts of reference calls, graph nodes, matched calls, missing calls, extra calls, correct edges, and incorrect edges. Define how calls are matched and compute node precision and recall when the reference permits it. Explain each discrepancy; fix the implementation or document a justified visibility limit.

Test the validator by changing a *copy* of one graph: remove a known call node or alter an edge, and show that the automated validator detects the error. Human inspection alone is not sufficient for the full-credit pathway.

Hint. Match calls by native call ID when possible. If no native ID exists, define and document a consistent matching rule. Check nodes first, then their attributes, and then the edges. Never modify the original evidence for the mutation test.

6.4 Apply the solution

After the three tasks work for one run, use the same process for the five required new runs: M0–M2, F1, and F2. At least one real run must contain two or more calls so that order can be demonstrated; add a run if necessary. Include a real repeated-call or retry case when you can reproduce one. Otherwise, use a clearly labeled synthetic fixture with two calls to the same tool to test that both calls remain separate nodes. The fixture does not replace a required real run.

For every real run, save the exact question, relevant tool setup, actual trace excerpt, independent reference record, machine-readable and readable graphs, validation report, final response, and a

short interpretation. An expected tool call that does not occur is still a valid observation; report no-tool, failed, blocked, repeated, or parallel behavior honestly.

Full-credit completion checklist

For the automated pathway, confirm that your team has:
- one documented command that produces a machine-readable graph, readable visualization, and validation report for a new saved run;
- an independent reference record and node/edge validation results for every required run;
- five required new runs and at least one real multi-call run;
- separate nodes for repeated calls and a repeated-call fixture if no real repetition was reproduced;
- one automated test that detects a deliberately altered graph;
- a data-flow diagram and short explanation of the design and alternative considered; and
- redacted shared evidence with unavailable fields marked unknown and no invented metadata.

7 Manual Fallback Option

The automated solution remains the expected and full-credit pathway. A team that makes a substantial automation attempt but cannot complete the automated graph pipeline may use this fallback without advance instructor approval. A fully correct fallback submission may earn at most **85/100**, equivalent to a solid B. The ceiling is not an automatic score: inaccurate graphs, missing runs, weak evidence, insufficient effort documentation, or unequal participation reduce the grade further.

7.1 Required manual work

Fallback teams must still:
- complete M0–M2 and F1–F2 as new real runs and include at least one real multi-call run;
- preserve the raw execution evidence and an independent reference record for every run;
- produce readable digital graphs with a diagram tool; informal hand sketches are insufficient;
- represent every actual call as a distinct node labeled with its call ID or order, tool name, outcome, and evidence pointer;
- define every edge meaning and avoid unsupported causal or total-order claims;
- provide a manual call inventory and node/edge comparison for every run, including reference, matched, missing, extra, correct-edge, and incorrect-edge counts, plus precision and recall when possible;
- explain all discrepancies, limitations, and no-tool, failed, blocked, repeated, or parallel behavior; and
- alter a copy of one graph and manually show how comparison with the reference reveals the introduced node or edge error.

Clearly label the report and repository as using the manual fallback. This pathway does not require a reusable trace-processing command, a machine-readable graph, or an automated validator. Partial automated components should still be submitted and demonstrated.

7.2 Required evidence of the automation attempt

To qualify for the 85-point ceiling, the report must summarize and the repository must preserve:
- the attempted collector, parser, graph generator, or validator code;
- a chronological troubleshooting record listing commands, errors, attempted fixes, and results;

- tests or saved outputs showing which components worked and which failed;
- a component-status table identifying completed, partial, and blocked features;
- a technical explanation of the unresolved blocker and reasonable next steps;
- Git commit history or equivalent dated evidence showing iterative work; and
- concrete implementation, testing, review, and documentation contributions from both students.

Documentation does not earn credit merely for length. It must demonstrate a good-faith technical attempt and make the reported failures reproducible.

8 Suggested Work Plan

1. Select Hermes or OpenClaw and run a small tool-using example.
2. Read the official tracing documentation and make a capture plan.
3. Define the event schema, graph node/edge semantics, and validation criteria.
4. Implement capture, storage, graph construction, and visualization.
5. Run the required scenarios, save evidence, and verify each graph.
6. Analyze omissions, false edges, failures, overhead, and reproducibility.
7. Prepare the report, presentation, and a live or recorded demonstration.

9 Final Technical Report

Prepare a clear, professional technical report in a standard single-column format. Use an 11- or 12-point readable font, 1-inch margins, numbered section headings, page numbers, figure and table captions, and consistent citations and references. The report may be created in Word, Google Docs, Overleaf, or another suitable editor, but it must be submitted as a PDF. The main report must be **at least 6 pages**, excluding references and an optional appendix. There is no strict maximum, but the report should remain focused and avoid unnecessary repetition. Include representative screenshots, graphs, and other essential run evidence in the main report; place additional supporting evidence in the optional appendix. Cite official framework documentation and any external code, datasets, or methods. Do not put API keys, private prompts, or sensitive tool outputs in the report source or submitted repository.

Use the following sections and subsections, adapting names only when the same information remains easy to locate:

1. **Title and team members**: state the project title, student names, selected platform, and model access path.
2. **Executive Summary**: summarize the method, main results, and most important limitations.
3. **Introduction**: motivation, problem statement, objectives, and contributions.
4. **Background and Related Work**: agent tool calling, chosen platform, existing trace facilities, and graph interpretation.
5. **Requirements and Design**: trace boundary, event schema, capture point, architecture, graph semantics, privacy decisions, and alternative approaches.
6. **Implementation**: platform/version, configuration, automated trace-to-graph command, generated outputs, reproducibility steps, and the team's GitHub repository URL with the commit hash used for evaluation. A fallback team instead presents its attempted implementation, component-status table, troubleshooting summary, unresolved blocker, and next steps.
7. **Experimental Method**: M0–M2 reruns and F1–F2, exact prompts, tools, model/settings, test conditions, reference record, matching rules, and metrics.

8. **Results and Verification**: trace/graph examples, per-run comparison table, correctness results, graph-mutation test, discrepancies, and any useful comparison with midterm observations. A fallback team clearly identifies its graphs and mutation check as manually produced and verified.
9. **Discussion and Limitations**: visibility gaps, concurrency or retries, threats to validity, and improvements.
10. **Conclusion**: what was achieved and what remains unresolved.
11. **References**: consistent citations to all external sources.
12. **Appendix (optional)**: extra graphs, detailed run records, prompts, setup instructions, and AI-assistance disclosure if too long for the main text.

Include a concise **team contributions and AI-assistance statement** in the report or appendix. Identify each student's concrete work and review activities. List the AI systems used, the tasks for which they were used (for example, design ideas, code, test generation, writing, or graph styling), and how the team checked their output. Using ChatGPT, Claude, or similar tools is **strongly recommended throughout the project**; students remain responsible for all submitted claims and code.

10 Deliverables and Submission

Each team must create its **own GitHub repository** for the project source code. Include the repository URL in the final report so a reader can find the exact implementation behind the reported results. Identify the evaluated commit hash there as well. The repository must be accessible to the instructor by the submission deadline.

Each student must independently submit **one Canvas assignment submission** containing:

1. The same final technical report PDF prepared by the team.
2. A share link to the editable report source (for example, Word, Google Docs, or Overleaf); grant the instructor access before submitting. A view-only PDF link is not sufficient.
3. The team's presentation slides (PPT or PPTX).
4. A link to the team's GitHub repository, with contents appropriate to the selected pathway:
   - **Automated pathway**: runnable processing command, dependency and setup instructions, redacted saved run evidence, machine-readable and rendered graphs, validation reports and tests, and the prompts and configuration needed to reproduce results.
   - **Manual fallback**: attempted code, troubleshooting record, component-status table, redacted run and reference evidence, editable graph sources and rendered graphs, manual comparisons, altered graph and detected discrepancy, and the prompts and configuration needed to reproduce results.

   The same URL and evaluated commit hash must appear in the report.
5. A short individual contribution statement, either in Canvas comments or as an attached file, consistent with the team statement in the report.

Both students submit the same team PDF, editable report-source link, slides, and repository link; each supplies their own contribution statement. The **oral presentation is required**. Both students must speak and be prepared to explain the design, code, validation method, and results. Demonstrate at least one question-to-trace-to-graph-to-verification sequence live or with a recorded backup. A fallback team demonstrates its manual graph construction and comparison and explains the attempted automation and unresolved blocker. Follow the instructor's Canvas announcement for the due date, presentation length, and submission link.

11 Evaluation Rubric (100 Points)

The following rubric applies to the full-credit automated pathway.

Criterion	Evidence expected	Points
Problem framing and design	Clear objective, platform choice, capture architecture, trace boundary, alternative considered, and justified graph semantics.	10
Automated trace processing	One documented command processes a new saved run and produces graph and validation outputs; unique call identity, accurate status/metadata, and correct handling of repeated, failed, and absent calls.	25
Visualization	Readable per-run graphs; distinct call nodes; meaningful labeled edges; status, question, and answer context; exported machine-readable and visual forms.	20
Testing and demonstration	M0–M2 reruns, F1–F2, a multi-call run, saved evidence, and reproducible execution.	10
Correctness verification	Independent reference; per-run node/edge checks and counts; precision/recall where possible; demonstrated graph-mutation test; explained discrepancies and limitations.	20
Technical report and reproducibility	Required report structure, clear results, consistent citations, setup/dependencies, privacy handling, contribution and AI-use disclosure.	10
Presentation and participation	Both students present, demonstrate one complete run, answer questions, and document substantially equal contributions.	5
Total		**100**

Manual fallback grading

The following maxima apply only to a submission that meets the eligibility and evidence requirements in Section 7.

Criterion	Maximum points
Problem framing and design	10/10
Automated trace processing and documented attempt	15/25
Visualization	20/20
Testing and demonstration	10/10
Manual correctness verification	15/20
Technical report and reproducibility	10/10
Presentation and participation	5/5
Maximum fallback total	**85/100**

The 15 automation points assess the attempted implementation, tests, iterative troubleshooting, partial working components, and reproducible failure evidence. The 15 verification points assess the accuracy and completeness of the manual comparisons and altered-graph check. The automated pathway is the only path to an A. The rubric awards credit for measured, honest results: a documented visibility gap or failed run can support a strong analysis, but an unsupported claim of perfect tracing cannot.

12 References and Official Starting Points

1. Nous Research, "Hermes Agent: Hooks," https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/features/hooks.md.
2. OpenClaw, "Trajectory bundles," https://docs.openclaw.ai/tools/trajectory.
3. OpenClaw, "Audit records," https://github.com/openclaw/openclaw/blob/main/docs/cli/audit.md.
4. OpenClaw, "OpenTelemetry," https://github.com/openclaw/openclaw/blob/main/docs/gateway/opentelemetry.md.
5. Nous Research, "Hermes Agent: Observer Hooks," https://github.com/NousResearch/hermes-agent/blob/main/website/docs/developer-guide/observer-hooks.md.
6. Nous Research, "Hermes Agent: Sessions," https://github.com/NousResearch/hermes-agent/blob/main/website/docs/user-guide/sessions.md.
7. Nous Research, "Hermes Agent: Plugins," https://github.com/NousResearch/hermes-agent/blob/main/website/docs/developer-guide/plugins/index.md.

Check the installed platform version against its corresponding documentation; tracing features and event fields can change.

A Example: One Possible Hermes Agent Tracking Solution

This appendix gives one possible design, not a required implementation or complete code submission. Students may use a different supported design. Check the installed Hermes version against its documentation before relying on the event names and fields below.

A.1 Architecture

Use a Hermes observer plugin as the graph's capture source and a separately exported Hermes session as the independent reference:

```
Hermes run
  |-- observer hooks --> hook_events.jsonl --> processor --> graph files
  `-- session store ----------------------> reference.jsonl
                                                    |
graph files + reference.jsonl ----------------> validator --> report
```

The graph generator reads only `hook_events.jsonl`. The validator separately reads the generated graph and `reference.jsonl`. Because the hook log and session export use different Hermes storage paths, the export can reveal a call missing from the hook log or graph. If the session export omits a blocked or cancelled call, document that visibility limit and supplement the reference with separate instrumentation around the tool boundary.

A.2 Step 1: Capture Hermes events

Create a native Hermes plugin, for example:

```
~/.hermes/plugins/tool-trace/
|-- plugin.yaml
`-- __init__.py
```

The manifest can declare four hooks:

```
name: tool-trace
version: 0.1.0
description: Save tool-call events for a course project
provides_hooks:
  - pre_llm_call
  - pre_tool_call
  - post_tool_call
  - post_llm_call
```

Register callbacks that accept `**kwargs` for forward compatibility. The following is an abbreviated sketch of the plugin core:

```
import hashlib, itertools, json, threading, time
from plugins.plugin_storage import plugin_data_dir

LOG = plugin_data_dir("tool-trace") / "hook_events.jsonl"
SEQ = itertools.count(1)
LOCK = threading.Lock()
FIELDS = (
    "session_id", "turn_id", "task_id", "api_request_id",
    "tool_call_id", "tool_name", "status", "duration_ms",
    "error_type", "error_message", "user_message",
    "assistant_response",
)

def digest(value):
    text = json.dumps(value, sort_keys=True, default=str)
    return hashlib.sha256(text.encode()).hexdigest()

def record(event, payload):
    with LOCK:
        row = {"event": event, "capture_seq": next(SEQ),
               "captured_at_ns": time.time_ns()}
        row.update({key: payload.get(key) for key in FIELDS})
        if "args" in payload:
            row["args_sha256"] = digest(payload["args"])
        if "result" in payload:
            row["result_sha256"] = digest(payload["result"])
        with LOG.open("a", encoding="utf-8") as handle:
            handle.write(json.dumps(row, default=str) + "\n")

def pre_llm(**kw):  record("pre_llm_call", kw)
def pre_tool(**kw): record("pre_tool_call", kw)
def post_tool(**kw): record("post_tool_call", kw)
def post_llm(**kw): record("post_llm_call", kw)

def register(ctx):
    ctx.register_hook("pre_llm_call", pre_llm)
    ctx.register_hook("pre_tool_call", pre_tool)
    ctx.register_hook("post_tool_call", post_tool)
    ctx.register_hook("post_llm_call", post_llm)
```

This sketch stores hashes rather than raw arguments and results. A team may store redacted values when its analysis needs them, but must not publish secrets or sensitive content. Record the plugin log path produced by the active Hermes profile; do not assume that every profile uses the default home directory.

Check and enable the plugin before collecting graded runs:

```
hermes plugins doctor ~/.hermes/plugins/tool-trace --ci
hermes plugins enable tool-trace
```

Test with one simple tool-using prompt. Confirm that one `pre_tool_call` and one `post_tool_call` share the same `tool_call_id`. A post event should provide the outcome, commonly `ok`, `error`, `blocked`, or `cancelled`. Preserve the raw JSONL before processing it.

A.3 Step 2: Construct the graph

The processor can treat one `turn_id` within a `session_id` as a run. It should:

1. select events for that run and retain their original line numbers as evidence pointers;
2. pair pre- and post-tool events by `tool_call_id`;
3. create one node per pre-tool event and attach the post event's status;
4. assign a documented generated ID only when Hermes supplies no call ID;
5. create edges labeled *next observed call* from the pre-event order; and
6. omit an ordering edge when the evidence shows concurrent calls and does not establish their order.

A node without a matching post event should remain in the graph with outcome *unknown* or *incomplete*; it must not be silently discarded. A repeated tool name still produces separate nodes. The processor should write both a machine-readable graph and a rendered graph, for example `graph.json`, `graph.dot`, and `graph.png`.

One possible graph record is:

```
{
  "run_id": "<session_id>:<turn_id>",
  "edge_semantics": "next observed call",
  "nodes": [
    {"id": "call_1", "tool": "read_file", "order": 1,
     "outcome": "ok", "evidence": "hook_events.jsonl:12"}
  ],
  "edges": []
}
```

A.4 Step 3: Create and use the independent reference

After the run, export the matching Hermes session from its persisted session store:

```
hermes sessions export reference.jsonl \
  --session-id SESSION_ID --redact
```

The reference parser should extract tool calls from assistant-message `tool_calls` and join later tool-result messages by `tool_call_id`. It must be separate from the hook-log parser. Match graph nodes to reference calls by native call ID when possible; otherwise use a documented combination of tool name, arguments or argument hash, and observed position.

For each run, the validator reports reference calls, graph nodes, matched calls, missing calls, extra calls, attribute disagreements, correct edges, and incorrect edges. Compute node precision as matched calls divided by graph nodes and node recall as matched calls divided by reference calls. If the export does not provide an independently checkable outcome or order, mark that check unavailable rather than claiming it passed.

The team-created command could have this interface:

```
python -m tracker process \
  --events runs/F1/hook_events.jsonl \
  --reference runs/F1/reference.jsonl \
  --run-id SESSION_ID:TURN_ID \
  --out runs/F1/output
```

It should produce the graph files and validation report in one run. To demonstrate the negative test, copy `graph.json`, remove one known node or change one edge, run the validator on the altered copy, and preserve the failing report. Never alter the raw hook log or independent reference.

A.5 Expected limitations

The team should test and report at least these limitations:

- A provider may omit a native `tool_call_id`; generated IDs then require an explicit matching rule.
- Concurrent calls may have an observable hook-emission order without a defensible causal order.
- The session export and observer log may expose different fields or boundary behavior.
- Hook callbacks can fail or time out, so missing hook events must remain detectable through the independent reference.
- Arguments, results, questions, answers, and local paths may require redaction before publication.